

Damas Inteligente: Agentes Adversariais com Minimax, Poda Alfa-Beta, MCTS e LLM em um Jogo de Damas Brasileiro

Pedro Lucas Santos Ferreira
Hícaro Vitor Teixeira de Abreu
pedro.l.ferreira@ufv.br
hicaros.abreu@ufv.br
Universidade Federal de Viçosa
Viçosa, Minas Gerais, Brasil

ABSTRACT

Este trabalho descreve um jogo de damas brasileiro com adversários de inteligência artificial, disponibilizado como aplicação web que roda inteiramente no navegador. Dois algoritmos foram implementados e colocados frente a frente: o Minimax com Poda Alfa-Beta, que percorre a árvore de jogadas para encontrar a melhor decisão, e a Busca em Árvore Monte Carlo (MCTS), que constrói seu julgamento a partir de milhares de partidas simuladas. O sistema tem ainda um Grande Modelo de Linguagem (LLM) como comentarista, traduzindo o estado do jogo em análises compreensíveis. Tudo foi feito em JavaScript, HTML e CSS, sem dependências externas. Para comparar os dois agentes, disputamos 200 partidas no modo IA contra IA, alternando as cores para que cada algoritmo começasse jogando metade das vezes. Com o Minimax buscando a profundidade 8 e o Monte Carlo usando 1.500 simulações por jogada, o Minimax venceu 139 partidas, o Monte Carlo venceu 57 e 4 terminaram empatadas. No tempo de resposta os dois praticamente empataram na média (cerca de 105 ms por jogada), mas com perfis bem diferentes: o Monte Carlo se manteve estável, entre 59 e 141 ms, enquanto o Minimax variou muito, indo de 23 ms em posições simples a mais de 700 ms nas mais complicadas. O trabalho mostra que, nessa configuração, o Minimax leva vantagem em força de jogo, e o Monte Carlo se destaca pela previsibilidade no tempo de resposta.

KEYWORDS

inteligência artificial, jogos adversariais, minimax, poda alfa-beta, Monte Carlo Tree Search, grandes modelos de linguagem, damas

1 INTRODUÇÃO

Jogos de tabuleiro para dois jogadores estão entre os domínios mais estudados em Inteligência Artificial (IA) desde os primeiros anos da área [5]. O apelo é simples: regras fixas, informação completamente visível para os dois lados e um resultado claro ao final. O jogo de damas brasileiro, jogado em tabuleiro 8×8 com 12 peças de cada lado, é mais difícil do que parece - o espaço de estados é estimado em 5×10^{20} posições [6], e qualquer tentativa de analisar tudo até o fim é computacionalmente inviável. Isso justifica o uso de heurísticas e poda.

O objetivo central deste trabalho foi desenvolver uma versão jogável de damas brasileiro em que o usuário pode enfrentar ou

apenas assistir dois agentes de IA: um baseado no algoritmo Minimax com Poda Alfa-Beta e outro na Busca em Árvore Monte Carlo (MCTS). A aplicação roda no navegador, sem servidor. O sistema tem também um Grande Modelo de Linguagem (LLM) como comentarista, atendendo ao requisito obrigatório da disciplina INF 420 - Inteligência Artificial I da Universidade Federal de Viçosa.

Escolhemos damas por razões práticas: o jogo é complexo o suficiente para exigir busca não trivial, mas tem literatura acadêmica bem estabelecida e regras claras. Ao colocar os dois algoritmos frente a frente, queríamos entender onde cada um se sai melhor. O Minimax busca com profundidade fixa e resultado determinístico; o MCTS distribui o esforço de forma adaptativa, priorizando as linhas que as simulações apontam como mais promissoras.

O artigo está organizado assim: a Seção 2 discute os trabalhos relacionados; a Seção 3 descreve a metodologia; a Seção 4 apresenta os resultados; a Seção 5 trata das questões éticas; e a Seção 6 conclui.

2 TRABALHOS RELACIONADOS

O jogo de damas tem história longa como campo de teste para IA. O trabalho de Schaeffer et al. [6] é provavelmente o mais citado: o programa Chinook resolveu o jogo de damas inglês, mostrando que o jogo perfeito de ambos os lados leva a empate. Para isso, o Chinook usava tabelas de finais de jogo pré-calculados combinadas com busca alfa-beta.

O algoritmo Minimax, formalizado por Shannon [7] e Von Neumann [8], é a base de quase todo agente competitivo em jogos de soma zero. A lógica é direta: um lado tenta maximizar o resultado, o outro tenta minimizá-lo. A Poda Alfa-Beta, de Knuth e Moore [3], tornou essa busca muito mais eficiente ao descartar ramos que não podem mudar a decisão final.

A Busca em Árvore Monte Carlo surgiu como alternativa ao Minimax em domínios com fator de ramificação alto ou sem boa função heurística [1]. Em vez de analisar posições até certa profundidade, o MCTS aprende sobre o tabuleiro rodando muitas partidas simuladas. A variante UCT, de Kocsis e Szepesvári [4], introduziu um critério de seleção baseado na fórmula $UCT = \bar{X}_j + C\sqrt{\ln n/n_j}$, que equilibra explorar jogadas novas e insistir nas que já deram resultado.

Nas damas, o Minimax tende a ser mais forte com boa função heurística; o MCTS leva vantagem quando a ramificação é alta ou o tempo por jogada é curto [1].

Mais recentemente, os LLMs passaram a ser integrados a sistemas de jogos, não para decidir jogadas, mas para narrar e interpretar o

que acontece. Modelos como GPT e Gemini foram explorados para descrever posições em linguagem natural e sugerir estratégias [2].

3 METODOLOGIA

3.1 Visão Geral do Sistema

O sistema é uma aplicação web em JavaScript puro, HTML5 e CSS3, que roda direto no navegador sem servidor ou bibliotecas externas. O código está disponível em <https://github.com/hicarop4/jogo-de-damas-com-minimax-e-mcst>. A tela mostra um tabuleiro 8×8 interativo com dois modos: Humano contra IA e IA contra IA.

As regras seguem o padrão oficial das Damas Brasileiras: movimentação diagonal, captura obrigatória pelo maior número de peças, promoção a dama ao atingir a última fileira adversária e movimentação livre da dama nas diagonais.

3.2 Minimax com Poda Alfa-Beta

O Minimax explora a árvore de jogadas até uma profundidade definida, alternando entre dois papéis: o maximizador (o agente) e o minimizador (o oponente). A Poda Alfa-Beta mantém dois limites, α (melhor valor garantido ao maximizador) e β (melhor valor garantido ao minimizador), e abandona qualquer ramo onde $\alpha \geq \beta$.

Para avaliar a posição s , a função heurística $h(s)$ usa:

- **Diferença de peças:** $w_1 \cdot (\text{peças}_{AI} - \text{peças}_{oponente})$
- **Diferença de damas:** $w_2 \cdot (\text{damas}_{AI} - \text{damas}_{oponente})$, com $w_2 > w_1$
- **Bônus de posição:** valoriza peças no centro e em posições mais protegidas

A profundidade é configurável, com padrão de 5 níveis. Posições terminais recebem valores extremos $\pm\infty$.

3.3 Busca em Árvore Monte Carlo (MCTS)

O MCTS repete quatro fases a cada decisão até esgotar o orçamento de simulações [1]:

- (1) **Seleção:** percorre a árvore escolhendo o nó filho com maior valor UCT até alcançar um nó ainda não expandido.
- (2) **Expansão:** se o nó não for terminal, uma jogada ainda não visitada é adicionada à árvore.
- (3) **Simulação (rollout):** a partir dali, joga-se com lances aleatórios até o fim, com limite de 200 lances para evitar ciclos.
- (4) **Retropropagação:** o resultado volta pelo caminho percorrido, atualizando contadores de visitas e vitórias.

A fórmula UCT usada é:

$$UCT(v_i) = \frac{w_i}{n_i} + C \cdot \sqrt{\frac{\ln N}{n_i}} \quad (1)$$

onde w_i é o número de vitórias do nó v_i , n_i é o total de visitas, N é o número de visitas ao nó pai e C é a constante de exploração ($C = \sqrt{2} \approx 1,4142$). Cada decisão usa 1.500 simulações, com limite de 200 lances por *rollout*.

3.4 Integração com LLM

Um Grande Modelo de Linguagem atua como comentarista. A cada jogada, o estado do tabuleiro é serializado e enviado ao modelo via API. O LLM devolve:

- uma avaliação da posição (quem está à frente, se há equilíbrio, ou quem está em desvantagem);
- um comentário sobre a última jogada;
- uma sugestão para o jogador humano, quando aplicável.

O LLM ficou restrito ao papel de comentarista porque chamadas à API introduzem latência imprevisível, incompatível com uma partida em tempo real. As limitações desse uso são discutidas na Seção 5.

3.5 Métricas de Avaliação

Para comparar os dois agentes no torneio de 200 partidas, usamos três medidas:

- **Taxa de vitória:** proporção de partidas ganhas por cada agente;
- **Número de lances por partida:** indica duração e equilíbrio das disputas;
- **Tempo médio por jogada:** custo computacional de cada decisão.

A taxa de vitória é a medida central, por mostrar diretamente qual agente joga melhor. O tempo por jogada mostra o custo de se ter essa qualidade. O número de lances diz se as partidas foram curtas e decididas rápido ou longas e disputadas até o fim.

4 RESULTADOS

4.1 Como o experimento foi feito

Os dados vêm de um torneio de 200 partidas no modo IA contra IA. Para que nenhum dos agentes fosse favorecido pela vantagem de começar jogando, alternamos as cores: em metade das partidas o Minimax jogou de Pretas e o Monte Carlo de Brancas, e na outra metade o contrário. Parâmetros: profundidade 8 para o Minimax; 1.500 simulações por jogada e $C = \sqrt{2}$ para o Monte Carlo. Um limite de 120 lances por partida evitou disputas sem fim.

4.2 Quem venceu o torneio

O Minimax ganhou 139 partidas, o Monte Carlo ganhou 57 e 4 terminaram empatadas (Figura 1). Em porcentagem: 69,5% para o Minimax e 28,5% para o Monte Carlo. Como as cores foram alternadas, essa vantagem não veio de quem começou jogando: o Minimax venceu 67 das 100 partidas em que jogou de Pretas e 72 das 100 em que jogou de Brancas. Buscar até a profundidade 8 deu ao Minimax uma leitura do tabuleiro que as 1.500 simulações do Monte Carlo não conseguiram acompanhar.

A Figura 2 mostra como a diferença se formou. As duas curvas saem juntas, mas o Minimax abre vantagem já nas primeiras dezenas de partidas e segue ampliando a distância até o fim. A inclinação quase constante das curvas indica que o domínio do Minimax foi parelho ao longo de todo o torneio, e não fruto de uma boa fase passageira.

4.3 Velocidade: empate na média, previsibilidade do Monte Carlo

Com 1.500 simulações por jogada, o Monte Carlo ficou tão custoso por lance quanto o Minimax: os dois gastaram cerca de 105 ms em média. A diferença está na regularidade. O Monte Carlo se manteve quase sempre na mesma faixa, entre 59 e 141 ms, porque roda um

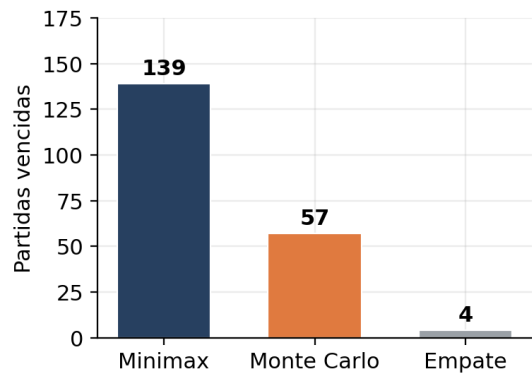


Figure 1: Total de partidas vencidas por cada agente no torneio de 200 jogos.

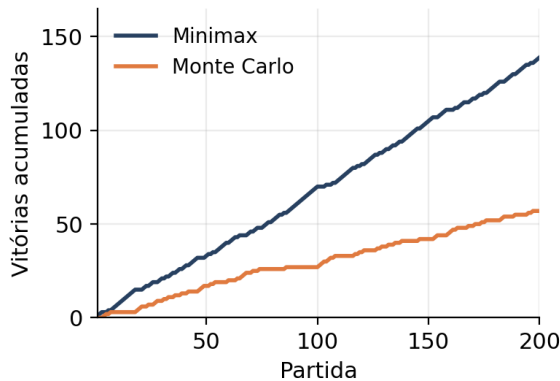


Figure 2: Vitórias acumuladas de cada agente ao longo das 200 partidas.

número fixo de simulações a cada jogada. Já o Minimax variou bastante (Figura 3): respondeu em cerca de 23 ms nas posições simples, mas em posições com muitas capturas e ramificações a busca de profundidade 8 chegou a passar de 700 ms. Para uma interface em tempo real, esse tempo previsível do Monte Carlo é uma vantagem prática, mesmo com a média igual.

No total das 200 partidas, o Minimax consumiu cerca de 923 segundos de processamento e o Monte Carlo cerca de 736 segundos. Como a média por jogada foi parecida, essa diferença no acumulado vem sobretudo dos picos do Minimax nas posições mais difíceis. A Tabela 1 reúne os principais números.

4.4 Duração das partidas

As partidas variaram bastante: a mais curta teve 39 lances, a mais longa atingiu o limite de 120 sem que nenhum dos agentes fechasse o adversário. A média ficou em torno de 73 lances. Das 200 disputas, 33 chegaram ao limite máximo, o que indica que parte dos jogos foi equilibrada demais para terminar dentro do tempo estipulado.

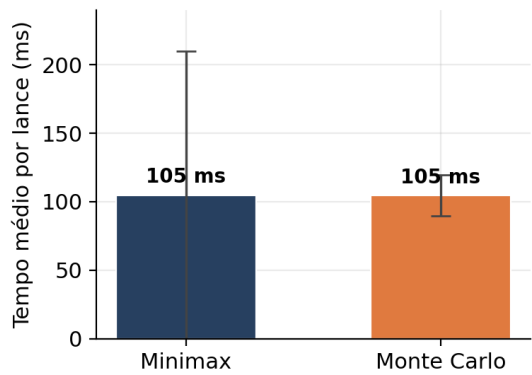


Figure 3: Tempo médio por lance de cada agente (a barra de erro indica a variação típica entre as jogadas).

Table 1: Resumo do torneio: Minimax (prof. 8) vs. Monte Carlo (1.500 simulações), em 200 partidas.

Medida	Minimax	Monte Carlo
Partidas vencidas	139	57
Taxa de vitória	69,5%	28,5%
Tempo médio por lance (ms)	104,8	104,6
Tempo mínimo por lance (ms)	22,8	58,7
Tempo máximo por lance (ms)	747,3	140,7
Tempo total no torneio (s)	≈923	≈736

5 CONSIDERAÇÕES ÉTICAS

5.1 Uso de Modelos Generativos

Usamos LLMs de duas formas neste projeto:

(1) **Apoio ao desenvolvimento:** durante a implementação, ferramentas baseadas em LLMs ajudaram a sugerir código, encontrar erros e gerar documentação. Toda sugestão foi revisada pelos autores antes de entrar no projeto.

(2) **Funcionalidade do sistema:** o comentarista usa um LLM externo via API para gerar análises em texto. O modelo utilizado foi Gemini Pro.

5.2 Limitações e Vieses dos LLMs

Usar LLMs como comentarista traz algumas limitações concretas:

- **Confiabilidade:** o modelo não garante que as avaliações estejam corretas, e um comentário errado pode induzir o jogador a uma decisão ruim;
- **Inconsistência:** por ser probabilístico, o modelo pode dar respostas contraditórias para posições similares;
- **Viés de treinamento:** o modelo provavelmente conhece melhor as damas inglesas do que as brasileiras, o que pode distorcer as análises;
- **Latência:** chamadas à API demoram um tempo variável, o que inviabiliza usar o LLM como jogador;
- **Privacidade:** os estados do jogo são enviados a um serviço externo, e isso precisa ficar claro ao usuário.

Por isso o LLM ficou restrito ao papel de comentarista, sem influência sobre as jogadas. As decisões ficam com os algoritmos, que sempre seguem as mesmas regras.

6 CONCLUSÃO E TRABALHOS FUTUROS

Construímos um jogo de damas brasileiro com dois adversários de IA - Minimax com Poda Alfa-Beta (profundidade 8) e Monte Carlo (UCT, $C = \sqrt{2}$, 1.500 simulações por lance) - mais um comentarista com LLM, tudo no navegador, sem servidor nem dependências externas.

O torneio de 200 partidas, com cores alternadas, mostrou um Minimax mais forte: 139 vitórias contra 57 do Monte Carlo, com 4 empates, e vantagem estável do começo ao fim. Buscar até a profundidade 8 fez diferença, e mesmo com 1.500 simulações por jogada o Monte Carlo não conseguiu acompanhar essa leitura mais profunda do tabuleiro - embora o aumento no número de simulações tenha reduzido a distância em relação a testes com menos simulações. No tempo de resposta, os dois ficaram parecidos na média (cerca de 105 ms por jogada), mas o Monte Carlo foi muito mais regular, enquanto o Minimax oscilou bastante e chegou a passar de 700 ms nas posições mais complexas. Em resumo: nessa configuração, o Minimax joga melhor, e o Monte Carlo entrega um tempo de resposta mais constante.

Os pontos fortes do projeto foram: rodar tudo no navegador sem servidor; combinar duas abordagens de busca com características opostas; e o comentarista com LLM, que torna o jogo mais acessível a quem não conhece IA. A maior dificuldade ficou no Minimax: na profundidade 8, o tempo de busca dispara em posições muito ramificadas, o que pode travar a interface em máquinas mais fracas.

O que aprendemos: a profundidade de busca pesa muito no Minimax, e na profundidade 8 ele supera um Monte Carlo de 1.500 simulações. Subir as simulações do Monte Carlo aproxima os dois, mas ao custo de mais tempo por jogada - tanto que, nesse ponto, o tempo médio dos dois já ficou parecido. Em compensação, o tempo do Monte Carlo é mais previsível, o que ajuda numa interface em tempo real. E, como antes, dá pra rodar um sistema de IA de porte médio direto no navegador.

Para trabalhos futuros, propomos: (1) adicionar tabelas de transposição e ordenação de jogadas ao Minimax, para que ele enxergue mais longe sem que o tempo dispare; (2) ajustar os pesos da função heurística com aprendizado por reforço; (3) testar variantes do Monte Carlo, como RAVE, e medir quantas simulações seriam necessárias para empatar com o Minimax de profundidade 8; (4) variar a profundidade do Minimax e o orçamento de simulações do Monte Carlo para mapear o ponto de equilíbrio entre os dois; e (5) avaliar o jogo com jogadores humanos de diferentes níveis.

ACKNOWLEDGMENTS

Os autores agradecem ao Prof. Julio Cesar Soares dos Reis pelas orientações durante o desenvolvimento do projeto, e à Universidade Federal de Viçosa pelo suporte acadêmico.

REFERENCES

- [1] Cameron Browne, Edward J. Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavenor, Diego Perez Liebana, Spyridon Samothrakis, and Simon Colton. 2012. A Survey of Monte Carlo Tree Search

- Methods. *IEEE Transactions on Computational Intelligence and AI in Games* 4, 1 (2012), 1–43. doi:10.1109/TCIAIG.2012.2186810
- [2] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, Harsha Nori, Hamid Palangi, Marco Tulio Ribeiro, and Yi Zhang. 2023. Sparks of Artificial General Intelligence: Early Experiments with GPT-4. arXiv:2303.12712 [cs.CL] doi:10.48550/arXiv.2303.12712
- [3] Donald E. Knuth and Ronald W. Moore. 1975. An Analysis of Alpha-Beta Pruning. *Artificial Intelligence* 6, 4 (1975), 293–326. doi:10.1016/0004-3702(75)90019-3
- [4] Levente Kocsis and Csaba Szepesvári. 2006. Bandit Based Monte-Carlo Planning. In *Proceedings of the 17th European Conference on Machine Learning (ECML 2006) (Lecture Notes in Computer Science, Vol. 4212)*. Springer, Berlin, Heidelberg, 282–293. doi:10.1007/11871842_29
- [5] Stuart J. Russell and Peter Norvig. 2016. *Artificial Intelligence: A Modern Approach* (3 ed.). Prentice Hall, Upper Saddle River, NJ.
- [6] Jonathan Schaeffer, Neil Burch, Yngvi Björnsson, Akihiro Kishimoto, Martin Müller, Robert Lake, Paul Lu, and Steve Sutphen. 2007. Checkers Is Solved. *Science* 317, 5844 (2007), 1518–1522. doi:10.1126/science.1144079
- [7] Claude E. Shannon. 1950. Programming a Computer for Playing Chess. *Philos. Mag.* 41, 314 (1950), 256–275. doi:10.1080/14786445008521796
- [8] John von Neumann and Oskar Morgenstern. 1944. *Theory of Games and Economic Behavior*. Princeton University Press, Princeton, NJ.